SERVICIO NACIONAL DE APRENDIZAJE SENA

Centro: Centro de Diseño e Innovación Tecnológica Industrial

Ficha: 3235906

Programa de Formación: TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE

Nombre del Aprendiz: Laura Barona Saavedra

Nombre del Instructor: Andrés Felipe Parra Martínez

Fecha: 12 de marzo 2026

INTRODUCCION

En el desarrollo de software moderno es fundamental aplicar buenas prácticas que permitan construir sistemas escalables, mantenibles y fáciles de entender. Para lograr esto, los desarrolladores utilizan patrones de software y arquitecturas de software que proporcionan soluciones probadas a problemas comunes en el diseño de aplicaciones.

En este documento se analizan conceptos clave como los patrones de software, la arquitectura de software y algunos tipos de arquitecturas utilizadas actualmente en la industria. Además, se explica la arquitectura seleccionada para el proyecto **Hogar360**, un sistema inmobiliario orientado a la gestión de propiedades, visitas y usuarios. Finalmente, se describen los diagramas de despliegue y de componentes aplicados al proyecto, los cuales permiten visualizar la estructura del sistema y cómo se distribuyen sus elementos.

¿Qué es un patrón de software?

Un **patrón de software** es una solución reutilizable y comprobada que se utiliza para resolver problemas comunes que aparecen durante el desarrollo de software. No se trata de código específico, sino de una guía o modelo que indica cómo estructurar una solución para un problema recurrente.

Los patrones de software ayudan a los desarrolladores a diseñar sistemas de manera más organizada, evitando reinventar soluciones y promoviendo buenas prácticas en la programación.

Algunos ejemplos de patrones de software son:

- **Singleton**: garantiza que una clase tenga una sola instancia.
- **Factory**: permite crear objetos sin especificar la clase exacta.
- **Observer**: permite que varios objetos reaccionen ante cambios en otro objeto.
- **MVC (Modelo Vista Controlador)**: separa la lógica del negocio, la interfaz y el control.

Los patrones son ampliamente utilizados porque facilitan la comunicación entre desarrolladores y permiten crear software más estructurado.

¿Por qué son importantes los patrones de software?

Los patrones de software son importantes porque aportan múltiples beneficios al proceso de desarrollo.

Primero, permiten **reutilizar soluciones ya probadas**, lo que reduce errores y acelera el desarrollo. En lugar de crear soluciones desde cero, los desarrolladores pueden aplicar patrones que ya han demostrado funcionar en muchos sistemas.

Segundo, los patrones mejoran la **organización del código**, haciendo que sea más fácil de mantener y modificar. Cuando el código sigue patrones conocidos, otros desarrolladores pueden comprenderlo más rápidamente.

También ayudan a mejorar la **escalabilidad y la calidad del software**, ya que promueven estructuras claras y bien definidas.

Finalmente, los patrones facilitan la **comunicación entre desarrolladores**, porque al mencionar un patrón específico se entiende rápidamente cómo funciona una parte del sistema.

¿Qué es una arquitectura de software?

La **arquitectura de software** es la estructura fundamental de un sistema de software. Define cómo se organizan los diferentes componentes de una aplicación, cómo se comunican entre sí y cómo se distribuyen las responsabilidades dentro del sistema.

La arquitectura sirve como un plano o guía para el desarrollo del software, permitiendo que el sistema sea más organizado, escalable y fácil de mantener.

Una buena arquitectura permite:

- Facilitar el mantenimiento del sistema.
- Mejorar la escalabilidad.
- Permitir la integración con otros sistemas.

- Separar responsabilidades dentro de la aplicación.

En proyectos grandes, la arquitectura es clave para asegurar que el sistema pueda evolucionar sin afectar su funcionamiento.

Tipos de arquitectura de software

Existen diferentes tipos de arquitecturas de software que se utilizan dependiendo de las necesidades del sistema.

Arquitectura Monolítica

En esta arquitectura toda la aplicación se desarrolla como una única unidad. Todos los módulos están integrados dentro del mismo sistema.

Ventajas:

- Fácil de desarrollar inicialmente.
- Menor complejidad de despliegue.

Desventajas:

- Difícil de escalar.
- Difícil mantenimiento en sistemas grandes.

Arquitectura en Capas

Divide el sistema en diferentes capas, cada una con una responsabilidad específica.

Ejemplo de capas:

- Presentación
- Lógica de negocio
- Acceso a datos

Ventajas:

- Organización clara del sistema.
- Facilita el mantenimiento.

Arquitectura de Microservicios

En esta arquitectura el sistema se divide en pequeños servicios independientes que se comunican entre sí mediante APIs.

Cada microservicio se encarga de una funcionalidad específica.

Ventajas:

- Alta escalabilidad.
- Permite desarrollar servicios de forma independiente.
- Facilita el mantenimiento.

Desventajas:

- Mayor complejidad de implementación.
- Requiere herramientas de comunicación entre servicios.

Arquitectura de Puertos y Adaptadores (Hexagonal)

Esta arquitectura busca separar completamente la lógica de negocio de las dependencias externas como bases de datos, APIs o interfaces de usuario.

Se basa en tres elementos principales:

- **Dominio (lógica del negocio)**
- **Puertos (interfaces)**
- **Adaptadores (implementaciones externas)**

Esto permite cambiar tecnologías sin afectar la lógica del negocio.

Arquitectura seleccionada para el proyecto

Para el proyecto **Hogar360** se seleccionó una combinación de:

- **Arquitectura de Microservicios**
- **Arquitectura de Puertos y Adaptadores (Hexagonal)**

Microservicios

El sistema se divide en varios microservicios, por ejemplo:

- Microservicio de usuarios
- Microservicio de propiedades
- Microservicio de visitas
- Microservicio de trazabilidad

Cada microservicio funciona de manera independiente y se comunica con los demás mediante APIs.

Puertos y Adaptadores

Dentro de cada microservicio se utiliza la arquitectura hexagonal para separar la lógica del negocio de las tecnologías externas.

Esto permite que:

- La lógica de negocio no dependa del framework.
- Se puedan cambiar bases de datos o APIs sin afectar el dominio.
- El código sea más mantenible y fácil de probar.

Esta combinación permite construir un sistema **escalable, modular y flexible**.

¿Qué es un diagrama de despliegue?

Un **diagrama de despliegue** es un tipo de diagrama que muestra cómo se distribuyen los componentes del sistema en la infraestructura física o virtual.

Este diagrama representa:

- Servidores
- Bases de datos
- Microservicios
- Redes

- Clientes o aplicaciones

El objetivo es mostrar **dónde se ejecuta cada parte del sistema y cómo se conectan entre sí**.

Es especialmente útil en arquitecturas de microservicios.

¿Qué es un diagrama de componentes?

Un **diagrama de componentes** muestra la organización de los componentes de software y cómo interactúan entre sí.

Este diagrama representa:

- Módulos del sistema
- Interfaces
- Dependencias entre componentes

Permite visualizar la estructura interna del software y entender cómo se comunican las diferentes partes del sistema.

Diagrama de Componentes del Proyecto Hogar 360

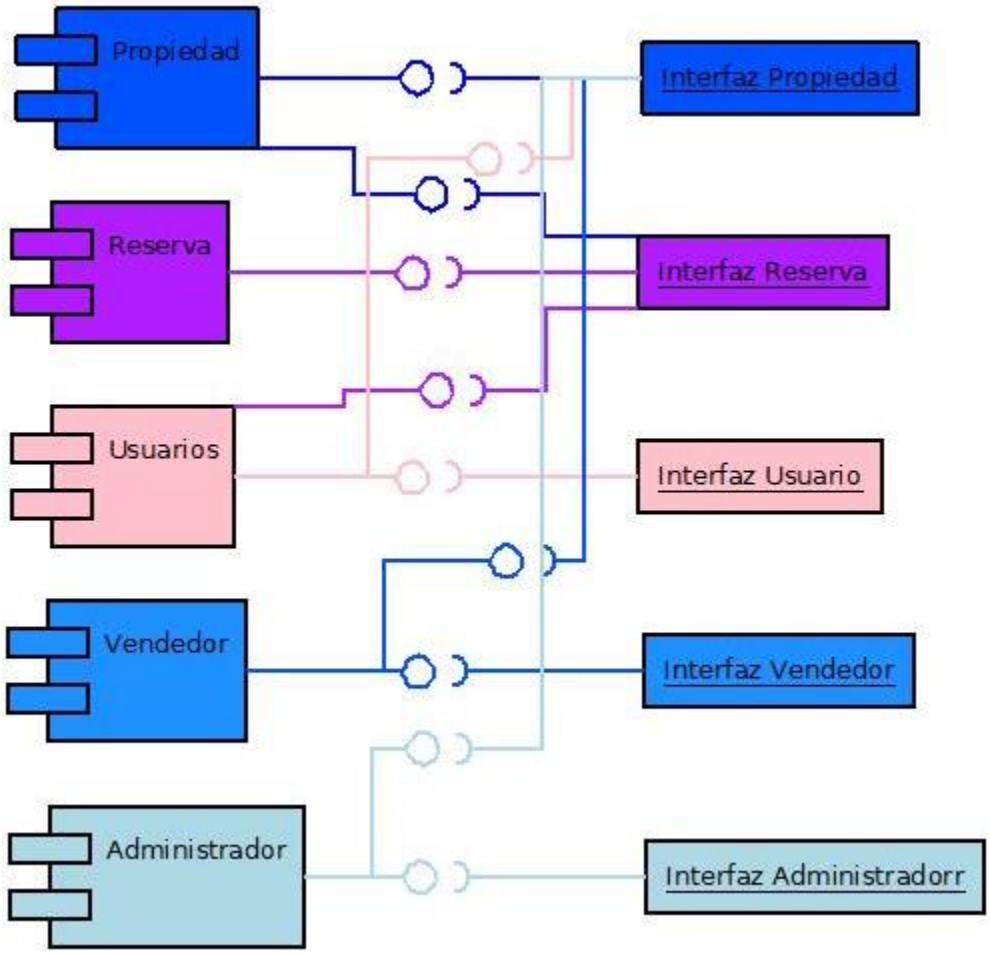
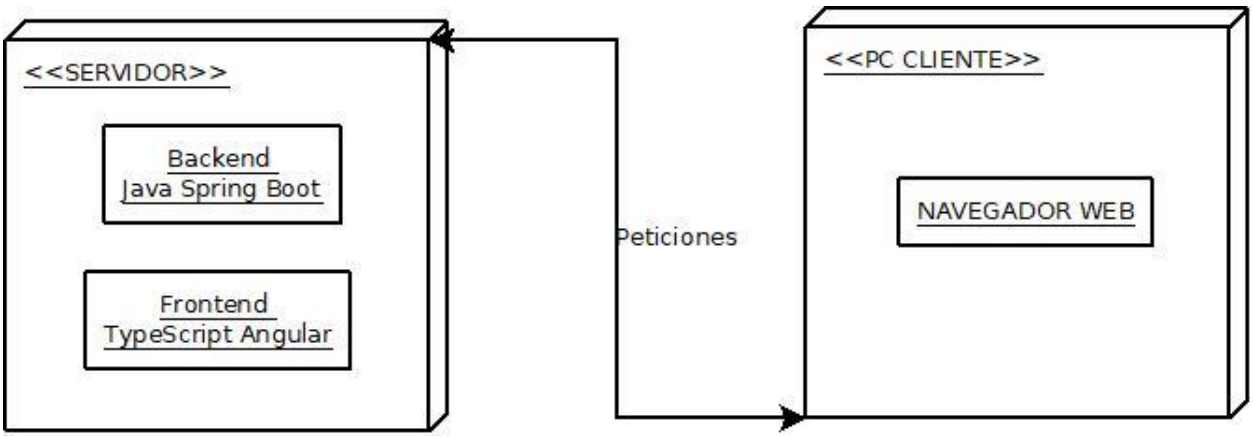


Diagrama de Despliegue de Hogar 360



CONCLUSIÓN

Los patrones y arquitecturas de software son elementos fundamentales en el desarrollo de sistemas modernos, ya que permiten estructurar aplicaciones de manera eficiente, reutilizable y escalable. Gracias a estos enfoques es posible crear software más organizado y fácil de mantener.

En el proyecto **Hogar360** se eligió una combinación de **arquitectura de microservicios y arquitectura hexagonal**, lo cual permite dividir el sistema en servicios independientes y mantener separada la lógica del negocio de las tecnologías externas. Esto facilita la escalabilidad, el mantenimiento y la evolución del sistema.

Además, los diagramas de despliegue y de componentes permiten visualizar la estructura del sistema tanto a nivel de infraestructura como de organización interna del software, facilitando la comprensión del funcionamiento general del proyecto.

En conclusión, aplicar correctamente patrones y arquitecturas de software contribuye significativamente a desarrollar sistemas robustos, flexibles y preparados para el crecimiento futuro.

REFERENCIAS

SENA. (s.f.). *Diseño de Patrones de Software* [Material de clase, documento proporcionado por la institución].